44

基于闪存的 SSD

经过几十年硬盘驱动器（hard disk drive）一统天下的局面，一种新型的持久存储（persistent storage）设备最近在世界上变得举足轻重。这类设备统称为**固态存储**（solid-state storage），它们不像硬盘那样有机械部件或运动部件，而是完全由晶体管（transistor）构成，这一点和内存、处理器很像。然而，和典型的随机存取存储器（random-access memory，例如 DRAM）不同，这种**固态存储设备**（solid-state storage device，又称 **SSD**）断电（power loss）后仍能保留信息，因此非常适合用来持久地存储数据。

我们要关注的技术叫作**闪存**（flash），更准确地说是**基于 NAND 的闪存**（NAND-based flash），它由 Fujio Masuoka 在 20 世纪 80 年代发明 [M+14]。我们会看到，闪存有一些独特的性质。例如，要写入其中某个单位（即**闪存页**，flash page），必须先擦除（erase）一个更大的单位（即**闪存块**，flash block），而擦除的代价可能相当高。此外，对同一页（page）写得太频繁，会让它**磨损**（wear out）。这两个性质让构建基于闪存的 SSD 成了一项有趣的挑战：

关键问题：如何构建基于闪存的 SSD
如何构建基于闪存的 SSD? 如何应对擦除代价高昂的问题? 既然反复覆写会让设备磨损（wear out），如何造出能长久使用的设备? 技术进步的脚步会停下来吗? 或者，它会不再令人惊叹吗?

44.1 存储单个位

闪存芯片（flash chip）的设计，是在单个晶体管（transistor）中存储一个或多个位；晶体管（transistor）中困住的电荷量被映射成一个二进制值。在**单层单元**（single-level cell，**SLC**）闪存中，一个晶体管（transistor）只存一个位（即 1 或 0）；而在**多层单元**（multi-level cell，**MLC**）闪存中，两个位被编码成不同的电荷水平，例如用低、偏低、偏高、高四个水平分别表示 00、01、10、11。甚至还有三**层单元**（triple-level cell，**TLC**）闪存，每个单元编码 3 个位。总的来说，SLC 芯片性能更高，价格也更贵。

当然，这种位级存储到底如何运作，深入到器件物理的层面，还有很多细节。这些超出了本书的范围，你可以自己去读更多资料 [J10]。

提示：术语要小心使用
你可能已经注意到，我们以前多次用过的一些术语，比如块（block）和页，也用在了闪存的语境下，只是用法和以前略有不同。新术语并不是为了故意给你添麻烦才造出来的（虽然效果可能正是如此），而是因为术语并没有一个中央权威来拍板。对你来说是块的东西，在别人那里可能是页，反之亦然，这取决于语境。你的任务很简单：了解每个领域里恰当的术语，用的时候让精通这门学科的人听得懂你在说什么。这种时候，唯一的办法简单却有时很痛苦：靠记性。

44.2 从位到 bank/平面

正如古希腊人所说，存一个位（或几个位），成不了存储系统。因此，闪存芯片（flash chip）被组织成 **bank** 或**平面**（plane），它们由大量单元组成。

访问一个 bank 时，有两种不同大小的单位：一种是**块**（block，有时也叫**擦除块**，erase block），大小通常是 128 KB 或 256 KB；另一种是**页**（page），大小为几 KB（例如 4 KB）。每个 bank 中有大量的块，每个块中又有大量的页。考虑闪存时，你必须记住这套新术语，它和我们在磁盘、RAID 中说的块，以及在虚拟内存（virtual memory）中说的页都不一样。

图 44.1 展示了一个包含块和页的闪存平面（flash plane）示例；在这个简单的例子中，有三个块，每个块包含四页。下面我们会看到为什么要区分块和页；事实证明，这一区别对读写之类的闪存操作至关重要，对设备的整体性能更是如此。你将学到的最重要（也最奇怪）的一点是：要写入块中的某一页，必须先擦除整个块。正是这个棘手的细节，让构建基于闪存的 SSD 成为一项有趣且值得做的挑战，这也是本章后半部分的主题。

44.3 闪存的基本操作

有了这样的闪存组织方式，闪存芯片（flash chip）支持三种底层操作。**读取**（read）命令用来从闪存中读出一页；**擦除**（erase）和**编程**（program）配合起来完成写入。具体如下：

- **读取（一页）：** 闪存芯片（flash chip）的客户端可以读取任何一页，只需向设备发出读取命令并指定相应的页号（page number）即可。这个操作通常相当快，大约几十微秒，与它在设备上的位置无关，而且（大体上）也与上一个请求的位置无关（这一点和磁盘截然不同）。能以同样快的速度访问任何位置，意味着这种设备是一种**随机访问**（random access）设备。
- **擦除（一个块）：** 由于设备的特性，在写入闪存中的某一页（page）之前，你必须先**擦除**（erase）这一页所在的整个块（block）。重要的是，擦除会破坏块中的内容（把每个位都置为 1）；因此，执行擦除之前，你必须确保块中你在乎的数据都已经复制到别



图 44.1：**一个简单的闪存芯片：块中的页**

处，比如内存或者另一个闪存块（flash block）中。擦除命令的代价相当高，需要几毫秒才能完成。擦除完成后，整个块被重置，其中每一页都可以编程了。

- **编程（一页）：** 块被擦除后，就可以用编程命令把页中的一些 1 改成 0，从而把一页所需的内容写入闪存。编程一页的代价比擦除一个块低，但比读取一页高，在现代闪存芯片（flash chip）上通常需要几百微秒左右。

可以这样理解闪存芯片（flash chip）：每一页都有一个与之关联的状态。页一开始处于 `INVALID`（无效）状态。擦除页所在的块之后，这一页（以及块中所有页）的状态就变为 `ERASED`（已擦除），这会重置块中每一页的内容，而且（重要的是）让它们变得可以编程。对一页编程之后，它的状态变为 `VALID`（有效），表示它的内容已经设定好，可以读取了。读取不影响这些状态（不过你应该只读取编程过的页）。一页一旦编程过，要改变它的内容，唯一的办法就是擦除它所在的整个块。下面这个例子展示了一个 4 页的块经过各种擦除和编程操作后的状态变化：

```
                  iiii  初始：块中各页都无效（i）
Erase()      →    EEEE  块中各页的状态被设为已擦除（E）
Program(0)   →    VEEE  编程第 0 页；状态被设为有效（V）
Program(0)   →    出错  页编程之后不能再次编程
Program(1)   →    VVEE  编程第 1 页
Erase()      →    EEEE  内容已擦除；所有页都可以编程
```

详细示例

写入的过程，也就是擦除和编程，非常特别，因此我们来看一个详细的例子，确保你弄懂了。在这个例子中，假设一个 4 页的块中有下面四个 8 位的页（页和块都小得不现实，但用在这个例子里很合适）；每一页之前都编程过，因此都处于 `VALID` 状态。

页 0	页 1	页 2	页 3
00011000	11001110	00000001	00111111
VALID	VALID	VALID	VALID

现在，假设我们要写第 0 页，填入新的内容。要写任何一页，都必须先擦除整个块。假设我们这样做了，块就处于这样的状态：

页 0	页 1	页 2	页 3
11111111	11111111	11111111	11111111
ERASED	ERASED	ERASED	ERASED

好消息！现在我们可以接着对第 0 页编程了，例如写入内容 `00000011`，按需要覆盖旧的第 0 页（内容为 `00011000`）。完成之后，块是这个样子：

页 0	页 1	页 2	页 3
00000011	11111111	11111111	11111111
VALID	ERASED	ERASED	ERASED

 © 2008–23, ARPACI-DUSSEAU

设备	读取 (µs)	编程 (µs)	擦除 (µs)
SLC	25	200–300	1500–2000
MLC	50	600–900	~3000
TLC	~75	~900–1350	~4500

图 44.2：**原始闪存的性能特征**

坏消息来了：第 1、2、3 页之前的内容全都没了！因此，覆写块内的任何一页之前，必须先把我们在乎的数据移到别的地方，例如内存或闪存上的其他位置。擦除的这种特性会深刻影响基于闪存的 SSD 的设计，我们很快就会看到。

小结

总的来说，读一页很容易：直接读就行。闪存芯片（flash chip）在这方面做得很好，速度也快；就性能而言，它们的随机读性能有可能远远超过现代磁盘驱动器（disk drive），后者因为机械寻道（seek）和旋转的开销而很慢。

写一页就麻烦一些：必须先擦除整个块，还要注意先把我们在乎的数据移到别处，然后再对所需的页编程。这样做不仅代价高，而且频繁重复这种编程/擦除循环（program/erase cycle），还会导致闪存芯片（flash chip）最大的可靠性（reliability）问题：**磨损**（wear out）。用闪存设计存储系统时，写入的性能和可靠性是关注的核心。我们很快会进一步了解现代 SSD 如何应对这些问题，在这些限制之下仍然提供出色的性能和可靠性。

44.4 闪存的性能和可靠性

我们关心的是用原始闪存芯片（flash chip）构建存储设备（storage device），因此有必要了解它们的基本性能特征。图 44.2 粗略汇总了大众媒体上的一些数据 [V12]。其中，作者给出了 SLC、MLC 和 TLC 闪存在读取、编程和擦除上的基本操作延迟；这三种闪存每个单元分别存储 1、2、3 位信息。

从表中可以看出，读取延迟相当低，只要几十微秒就能完成。编程延迟更高，波动也更大，SLC 低至 200 微秒，但每个单元塞进的位越多，延迟就越高；要获得良好的写入性能，就得并行地（in parallel）使用多个闪存芯片（flash chip）。最后，擦除的代价相当高，通常需要几毫秒。如何应对这一开销，是现代闪存存储设计的核心问题。

现在来考虑闪存芯片（flash chip）的可靠性。机械磁盘可能因为各种各样的原因发生故障（包括可怕而又十分“物理”的**磁头碰撞**（head crash），即磁头实际接触到了记录面）；而闪存芯片（flash chip）是纯硅的，从这个意义上说，需要担心的可靠性问题要少一些。主要的问题是**磨损**（wear out）：闪存块（flash block）每擦除和编程一次，都会慢慢积累一点额外的电荷。随着时间推移，额外的电荷越积越多，区分 0 和 1 就越来越难。到了无法区分的地步，这个块就不能用了。

块的典型寿命目前还不是很清楚。厂商标称基于 MLC 的块寿命为 10,000 次 P/E（program/erase，编程/擦除）周期，也就是说，每个块在失效之前可以擦除和编程 10,000 次。

提示：向后兼容的重要性
在分层的系统中，**向后兼容**（backwards compatibility）始终是需要考虑的问题。在两个系统之间定义稳定的接口，就能在保证双方持续互通的同时，让接口两边各自创新。这种做法在许多领域都相当成功：操作系统为应用程序提供相对稳定的 API，磁盘为文件系统（file system）提供同样的基于块的接口，IP 网络协议栈（protocol stack）的每一层都为上一层提供固定不变的接口。
不出所料，这种僵化也有缺点：上一代定义的接口未必适合下一代。在某些情况下，考虑把整个系统彻底重新设计一遍可能是有益的。Sun 的 ZFS 文件系统就是一个绝佳的例子 [B07]：ZFS 的设计者重新审视了文件系统和 RAID 之间的交互，构想出（并最终实现了）一个更高效的整体。

基于 SLC 的芯片每个晶体管（transistor）只存一个位，因此标称寿命更长，通常是 100,000 次 P/E 周期。不过，最近的研究表明，实际寿命比预期的长得多 [BD10]。

闪存芯片（flash chip）的另一个可靠性问题叫作**干扰**（disturbance）。访问闪存中的某一页时，相邻页中的一些位可能会发生翻转；根据这一页是在被读取还是在被编程，这种位翻转分别称为**读干扰**（read disturb）或**编程干扰**（program disturb）。

44.5 从原始闪存到基于闪存的 SSD

对闪存芯片（flash chip）有了基本了解之后，我们面临下一个任务：如何把一组基本的闪存芯片变成看起来像典型存储设备的东西。标准的存储接口是一个简单的基于块的接口：给定块地址，就可以读写大小为 512 字节（或更大）的块，也就是扇区（sector）。基于闪存的 SSD 的任务，就是在它内部的原始闪存芯片（flash chip）之上，提供这种标准的块接口。

在内部，SSD 由若干闪存芯片（flash chip）组成，用于持久存储。SSD 还包含一定数量的易失性（volatile，即非持久的）内存（例如 SRAM）；这种内存可以用来缓存（cache）和缓冲数据，也可以存放映射表（mapping table），我们下面会讲到。最后，SSD 还包含控制逻辑，用来协调设备的各项操作。细节请参阅 Agrawal 等人的论文 [A+08]；简化的框图见图 44.3（第 6 页）。

这一控制逻辑的基本功能之一，是满足客户端的读写请求，并根据需要把它们转换成内部的闪存操作。**闪存转换层**（flash translation layer，**FTL**）提供的正是这一功能。FTL 接收针对*逻辑块*（logical block，它们构成设备接口）的读写请求，把它们转换成对底层*物理块*（physical block）和*物理页*（physical page）的底层读取、擦除和编程命令；物理块和物理页构成了实际的闪存设备。FTL 完成这项任务时，应当以提供出色的性能和高可靠性为目标。

我们会看到，出色的性能可以通过组合多种技术来实现。关键之一是**并行**（in parallel）使用多个闪存芯片（flash chip）；虽然我们不会再进一步讨论这一技术，但只需知道，所有现代 SSD 内部都使用多个芯片来获得更高的性能。另一个性能目标是减少**写放大**（write amplification），它的定义是：FTL 向闪存芯片（flash chip）发出的写流量总量（以字节计），

 © 2008–23, ARPACI-DUSSEAU

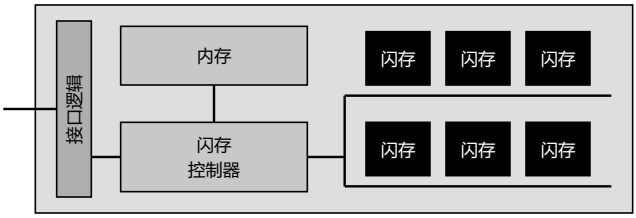


图 44.3：**基于闪存的 SSD：逻辑框图**

除以客户端向 SSD 发出的写流量总量（以字节计）。我们下面会看到，构建 FTL 的幼稚方法会导致很高的写放大（write amplification）和很低的性能。

高可靠性则要结合几种不同的方法来实现。如前所述，一个主要问题是**磨损**（wear out）。如果某个块被擦除和编程得太频繁，它就会无法使用；因此，FTL 应该尽量把写操作均匀地分散到闪存的各个块上，确保设备中所有块大致在同一时间磨损（wear out）；这种做法叫作**磨损均衡**（wear leveling），是所有现代 FTL 必不可少的组成部分。

另一个可靠性问题是编程干扰（program disturb）。为了尽量减少这种干扰，FTL 通常会在已擦除的块中按顺序编程，从低页号（page number）到高页号。这种顺序编程的方法能把干扰降到最低，被广泛采用。

44.6 FTL 的组织方式：一种糟糕的方法

FTL 最简单的组织方式，我们称之为**直接映射**（direct mapped）。在这种方法中，对逻辑页（logical page） N 的读取直接映射为对物理页 N 的读取。对逻辑页（logical page） N 的写入就复杂一些了：FTL 首先要读入页 N 所在的整个块，然后擦除这个块，最后对旧页和新页编程。

你大概能猜到，直接映射（direct-mapped）的 FTL 在性能和可靠性两方面都有很多问题。性能问题出在每一次写入上：设备必须读入整个块，擦除它（代价相当高），再对它编程。最终结果是严重的写放大（write amplification，与一个块中的页数成正比），写性能因此极差，甚至比有机械寻道和旋转延迟（rotational delay）的普通硬盘还慢。

这种方法的可靠性就更糟了。如果文件系统元数据（metadata）或用户文件数据被反复覆写，同一个块就会被一遍又一遍地擦除和编程，很快就会磨损（wear out），还可能丢失数据。直接映射（direct-mapped）的方法把磨损（wear out）的控制权过多地交给了客户端的工作负载（workload）；如果工作负载没有把写负载均匀地分散到各个逻辑块上，存放热门数据的底层物理块就会很快磨损（wear out）。无论出于可靠性还是性能的考虑，直接映射（direct-mapped）的 FTL 都是个坏主意。

44.7 日志结构的 FTL

出于这些原因，如今大多数 FTL 都是**日志结构**（log structured）的。这种思想既适用于存储设备（我们现在就会看到），也适用于构建在它们之上的文件系统（例如**日志结构文件系统**，log-structured file system）。写入逻辑块 N 时，设备把这次写入追加到当前正在

写入的块中的下一个空闲位置；我们把这种写入方式称为**日志**（logging）。为了支持之后对块 *N* 的读取，设备会维护一张**映射表**（mapping table，保存在内存中，同时以某种形式持久地保存在设备上）；这张表记录了系统中每个逻辑块的物理地址（physical address）。

我们通过一个例子，来确认自己理解了这种基本的基于日志的方法是如何工作的。对客户端来说，这个设备看起来就像一块普通的磁盘，可以读写 512 字节的扇区或扇区组。为简单起见，假设客户端以 4 KB 为单位读写。再假设 SSD 包含大量 16 KB 大小的块，每个块分成四个 4 KB 的页；这些参数并不现实，因为闪存块（flash block）通常包含更多的页，但很适合用来讲解。

假设客户端发出如下操作序列：

- Write(100)，内容为 `a1`
- Write(101)，内容为 `a2`
- Write(2000)，内容为 `b1`
- Write(2001)，内容为 `b2`

这些**逻辑块地址**（logical block address，例如 100）由 SSD 的客户端，例如文件系统，用来记住信息存放在哪里。

在内部，设备必须把这些块写入转换成原始硬件支持的擦除和编程操作，并以某种方式为每个逻辑块地址（logical block address）记录：SSD 的哪个**物理页**（physical page）存放着它的数据。假设 SSD 的所有块当前都无效，在对任何页编程之前都必须先擦除。下面是 SSD 的初始状态，所有页都标记为 `INVALID (i)`：

块:	0				1				2			
页:	00	01	02	03	04	05	06	07	08	09	10	11
内容:												
状态:	i	i	i	i	i	i	i	i	i	i	i	i

SSD 收到第一个写请求，即写逻辑块 100 时，FTL 决定把它写到物理块 0；物理块 0 包含四个物理页：0、1、2、3。因为这个块还没有擦除，所以暂时不能写入；设备必须先向块 0 发出擦除命令。这样就得到下面的状态：

块:	0				1				2			
页:	00	01	02	03	04	05	06	07	08	09	10	11
内容:												
状态:	E	E	E	E	i	i	i	i	i	i	i	i

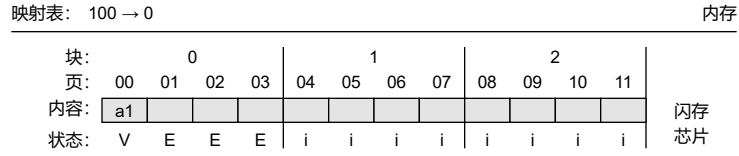
块 0 现在可以编程了。大多数 SSD 会按顺序（即从低到高）写页，以减少与**编程干扰**（program disturb）相关的可靠性问题。接着，SSD 把逻辑块 100 的写入导向物理页 0：

块:	0				1				2			
页:	00	01	02	03	04	05	06	07	08	09	10	11
内容:	a1											
状态:	V	E	E	E	i	i	i	i	i	i	i	i

但如果客户端想读逻辑块 100 呢？它怎么找到数据在哪里？SSD 必须把对逻辑块 100 的读取转换成对物理页 0 的读取。为了支持这一功能，FTL 把逻辑块 100 写入物理页 0 时，

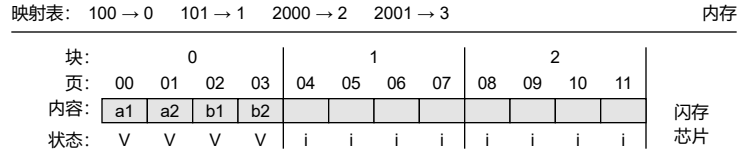
 © 2008–23, ARPACI-DUSSEAU

会把这一事实记录在一张**内存中的映射表**（in-memory mapping table）里。我们也会在示意图中跟踪这张映射表（mapping table）的状态：



现在你可以看到客户端写 SSD 时会发生什么了。SSD 为这次写入找一个位置，通常就是选下一个空闲页；然后用块的内容对这一页编程，并在映射表（mapping table）中记录逻辑到物理的映射。之后的读取只需借助这张表，把客户端给出的逻辑块地址（logical block address）**转换**（translate）成读取数据所需的物理页号即可。

我们来看示例写入流中剩下的写操作：101、2000 和 2001。写完这些块后，设备的状态如下：



基于日志的方法天然就能提高性能：只是偶尔才需要擦除，并且完全避免了直接映射（direct-mapped）方法中代价高昂的“读取—修改—写入”；它还能大大提升可靠性。FTL 现在可以把写操作分散到所有页上，也就是进行所谓的**磨损均衡**（wear leveling），从而延长设备的寿命；我们会在后面进一步讨论磨损均衡（wear leveling）。

可惜的是，这种基本的日志结构（log-structured）方法也有一些缺点。第一，覆写逻辑块会产生我们所说的**垃圾**（garbage），也就是散落在设备各处、占用空间的旧版本数据。设备必须定期执行**垃圾回收**（garbage collection，**GC**），找出这些块并为将来的写入腾出空间；过多的垃圾回收会推高写放大（write amplification），降低性能。第二，内存中的映

补充：FTL 映射信息的持久化

你可能在想：如果设备断电了会怎样？内存中的映射表（mapping table）会消失吗？显然，这些信息不能真的丢失，否则设备就无法作为持久存储设备工作了。SSD 必须有办法恢复映射信息。

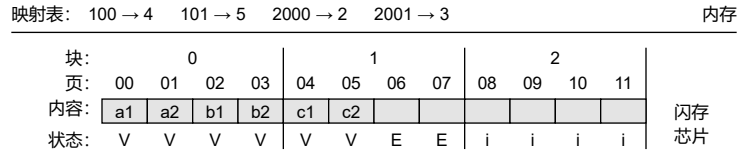
最简单的做法，是在每一页上记录一些映射信息，记在所谓的**带外**（out-of-band，**OOB**）区域中。设备断电并重启后，必须扫描各个 OOB 区域，在内存中重建映射表（mapping table）。这种基本方法有它的问题：扫描一块大容量 SSD 来找出所有必要的映射信息，速度很慢。为了克服这一局限，一些高端设备使用更复杂的**日志**（logging）和**检查点**（checkpointing）技术来加快恢复；要了解日志（logging）的更多内容，请阅读有关崩溃一致性（crash consistency）和日志结构文件系统（log-structured file system）的章节 [AD14a]。

射表（mapping table）代价很高；设备越大，这些表需要的内存就越多。下面我们依次讨论这两个问题。

44.8 垃圾回收

像这样的日志结构（log-structured）方法，第一个代价是会产生垃圾，因此必须进行**垃圾回收**（garbage collection），即回收无效块（dead block）。我们用前面的例子来说明这一点。回想一下，逻辑块 100、101、2000 和 2001 已经写入了设备。

现在假设块 100 和 101 再次被写入，内容分别是 `c1` 和 `c2`。这些写入被写到下一个空闲页，在本例中是物理页 4 和 5；映射表（mapping table）也相应更新。注意，设备必须先擦除块 1，才能进行这样的编程：



现在的问题应该很明显了：物理页 0 和 1 虽然标记为 `VALID`，里面存的却是**垃圾**（garbage），即块 100 和 101 的旧版本。由于设备是日志结构（log-structured）的，覆写会产生垃圾块（garbage block），设备必须回收它们，才能为新的写入提供空闲空间（free space）。

找出垃圾块（garbage block，也称**无效块**，dead block）并回收以供将来使用的过程，叫作**垃圾回收**（garbage collection），它是所有现代 SSD 的重要组成部分。基本过程很简单：找到一个包含一个或多个垃圾页（garbage page）的块，从中读出有效（非垃圾）页，把这些有效页写到日志（log）中，最后回收整个块，用于写入。

下面举例说明。设备决定回收上面块 0 中的无效页（dead page）。块 0 中有两个无效页（页 0 和 1）和两个有效页（页 2 和 3，分别存放着块 2000 和 2001）。为此，设备将：

- 从块 0 读出有效数据（页 2 和 3）
- 把有效数据写到日志末尾
- 擦除块 0（释放它，以供之后使用）

垃圾回收器（garbage collector）要能工作，每个块中都必须有足够的信息，让 SSD 判断每一页是有效还是无效。实现这一点的一种自然方法，是在每个块中的某个位置，存放每一页中存的是哪个逻辑块的信息。然后，设备就可以利用映射表（mapping table），判断块中的每一页是否存放着有效数据。

在上面的例子中，垃圾回收发生之前，块 0 存放着逻辑块 100、101、2000、2001。这时映射表（mapping table）中是 100→4、101→5、2000→2、2001→3，通过检查映射表，设备很容易判断这个 SSD 块中的每一页是否存放着有效信息。例如，页 2 和 3 显然仍被映射表（mapping table）指向；页 0 和 1 则没有，因此是垃圾回收的候选对象。

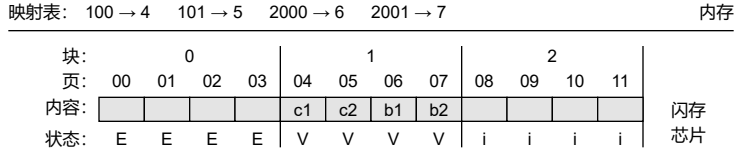
在我们的例子中，这次垃圾回收完成后，设备的状态如下：

补充：一种新的存储 API：trim

说到硬盘，我们通常只会想到读写它们最基本的接口：读和写（通常还有某种**缓存刷新**（cache flush）命令，确保写入的数据真正持久化了，不过为简单起见，我们有时会略去它）。对日志结构（log-structured）的 SSD，乃至任何维护着灵活多变的逻辑到物理块映射（logical-to-physical block mapping）的设备来说，有一个新接口很有用，叫作 **trim** 操作。

trim 操作接收一个地址（可能还有一个长度），只是告诉设备：这个地址（和长度）所指定的块已被删除；这样，设备就不必再记录与这一地址范围有关的任何信息了。对标准硬盘来说，trim 并不是特别有用，因为硬盘中块地址（block address）到具体盘片（platter）、磁道（track）和扇区的映射是静态的。然而，对日志结构（log-structured）的 SSD 来说，知道某个块不再需要了是非常有用的，因为 SSD 可以从 FTL 中删除这条信息，之后在垃圾回收时再回收这块物理空间。

虽然我们有时会把接口和实现看作两个独立的东西，但在这个例子中，我们看到是实现塑造了接口。在映射复杂的情况下，知道哪些块不再需要，就能让实现更高效。



可以看到，垃圾回收的代价可能很高，需要读取并重写有效数据。最理想的回收对象，是只包含无效页（dead page）的块；这样的块可以立即擦除，用来存放新数据，无需代价高昂的数据迁移（migration）。

为了降低 GC 的开销，一些 SSD 会为设备**预留空间**（overprovision）[A+08]：增加额外的闪存容量之后，清理工作就可以推迟，放到**后台**（background）进行，也许挑设备不太忙的时候做。增加容量还能提高内部带宽（bandwidth），这些带宽可以用于清理，从而不影响客户端感知到的带宽。许多现代 SSD 都以这种方式预留空间（over-provisioning），这是获得出色整体性能的关键之一。

44.9 映射表的大小

日志结构（log-structured）的第二个代价是映射表（mapping table）可能非常大，设备的每个 4 KB 页都要占一项。例如，对于一块 1 TB 的大容量 SSD，每个 4 KB 页一个 4 字节的表项，仅这些映射就需要设备拥有 1 GB 内存！因此，这种**页级**（page-level）FTL 方案是不切实际的。

基于块的映射

降低映射开销的一种方法，是对设备的每个块（block）只保存一个指针，而不是每页一个，这样映射信息的量就缩小为原来的 $1/\left(\frac{Size_{block}}{Size_{page}}\right)$。这种**块级**（block-level）FTL 类似于

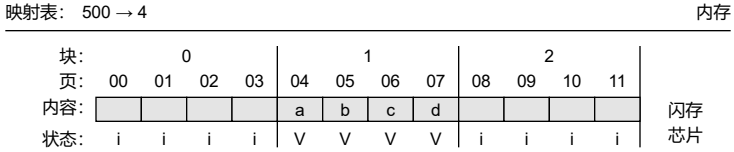
在虚拟内存系统中使用更大的页：那样的话，VPN 用的位数更少，每个虚拟地址（virtual address）中的偏移量（offset）更大。

可惜的是，在基于日志的 FTL 中，基于块的映射在性能上表现并不好。最大的问题出现在“小写入”时（即写入量小于一个物理块的大小）。这种情况下，FTL 必须从旧块中读出大量有效数据，把它们（连同这次小写入的数据）复制到一个新块中。这种数据复制会大大增加写放大（write amplification），从而降低性能。

为了把这个问题说得更清楚，我们来看一个例子。假设客户端之前写入了逻辑块 2000、2001、2002 和 2003（内容分别为 `a`、`b`、`c`、`d`），它们位于物理块 1 中的物理页 4、5、6、7。如果按页映射，转换表必须为这些逻辑块记录四个映射：2000→4、2001→5、2002→6、2003→7。

如果改用块级（block-level）映射，FTL 只需要为所有这些数据记录一个地址转换（address translation）。不过，这里的地址映射和前面的例子略有不同。具体来说，我们把设备的逻辑地址空间（logical address space）看作被切成了一个个 chunk，每个 chunk 的大小和闪存中物理块的大小相同。这样，逻辑块地址（logical block address）就由两部分组成：chunk 号和偏移量。由于我们假设每个物理块能放下四个逻辑块，逻辑地址（logical address）的偏移量部分需要 2 位；其余的（最高的）那些位构成 chunk 号。

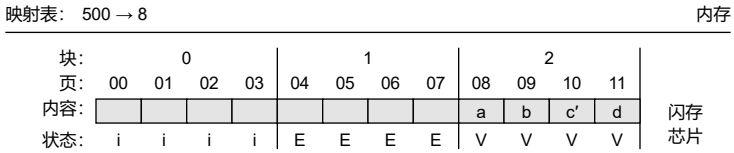
逻辑块 2000、2001、2002 和 2003 的 chunk 号都相同（500），偏移量各不相同（分别是 0、1、2、3）。因此，使用块级（block-level）映射时，FTL 记录的是 chunk 500 映射到块 1（从物理页 4 开始），如下图所示：



在基于块的 FTL 中，读取很容易。首先，FTL 从客户端给出的逻辑块地址（logical block address）中取出最高的几位，得到 chunk 号。然后，FTL 在表中查找 chunk 号到物理页的映射。最后，FTL 把逻辑地址（logical address）中的偏移量加到块的物理地址上，算出所需闪存页（flash page）的地址。

例如，如果客户端读取逻辑地址（logical address） 2002，设备会取出逻辑 chunk 号（500），在映射表（mapping table）中查找对应的转换（得到 4），再把逻辑地址中的偏移量（2）加到这个转换结果（4）上。得到的物理页地址（6）就是数据所在的位置；FTL 随后就可以向这个物理地址发出读取，得到所需的数据（`c`）。

但如果客户端写逻辑块 2002（内容为 `c'`）呢？这种情况下，FTL 必须读入 2000、2001 和 2003，然后把四个逻辑块全部写到一个新位置，并相应地更新映射表（mapping table）。之后，块 1（原来存放数据的地方）就可以擦除并重新使用了，如下所示：



 © 2008–23, ARPACI-DUSSEAU

从这个例子可以看出，块级（block-level）映射虽然大大减少了转换所需的内存，但当写入小于设备的物理块大小时，会导致严重的性能问题；由于实际的物理块可能有 256 KB 甚至更大，这样的写入很可能会经常发生。因此，需要更好的解决方案。你感觉到了吗？本章接下来就要告诉你这个方案是什么了。更好的是，在继续往下读之前，你能自己想出来吗？

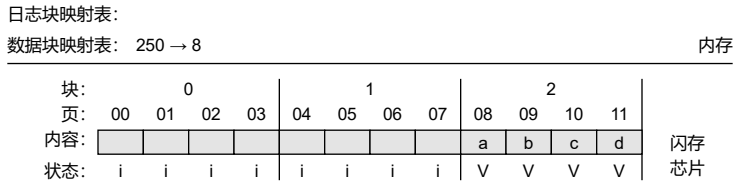
混合映射

为了既能灵活地写入，又能降低映射开销，许多现代 FTL 采用了**混合映射**（hybrid mapping）技术。采用这种方法时，FTL 会保留几个已擦除的块，把所有写入都导向这些块；它们称为**日志块**（log block）。由于 FTL 希望能把任何一页写到日志块（log block）中的任何位置，而不必像纯粹的块映射那样做大量复制，因此它为这些日志块保留逐页的映射。

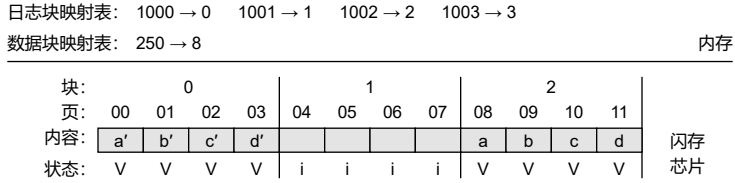
因此，FTL 的内存中在逻辑上有两类映射表（mapping table）：一小组逐页映射，放在我们所说的*日志块映射表*（log table）中；一大组逐块映射，放在*数据块映射表*（data table）中。查找某个逻辑块时，FTL 会先查日志块映射表（log table）；如果在那里找不到这个逻辑块的位置，FTL 再查数据块映射表（data table）找到它的位置，然后访问所请求的数据。

混合映射（hybrid mapping）策略的关键，是把日志块（log block）的数量控制得很少。为此，FTL 必须定期检查日志块（它们每页一个指针），把它们转换成只需一个块指针就能指向的块。根据块的内容不同，这种转换主要通过三种技术之一来完成 [KK+02]。

例如，假设 FTL 之前写入了逻辑页（logical page） 1000、1001、1002 和 1003，把它们放在物理块 2 的物理页 8、9、10、11 中；假设写入 1000、1001、1002 和 1003 的内容分别是 `a`、`b`、`c` 和 `d`。

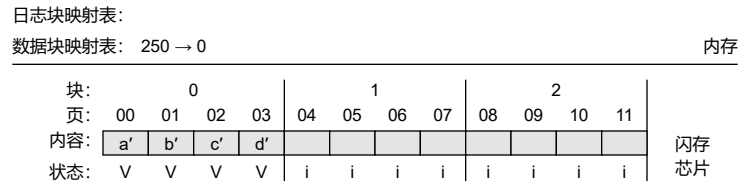


现在假设客户端以完全相同的顺序覆写这几个块（数据为 `a'`、`b'`、`c'` 和 `d'`），写到当前可用的一个日志块（log block）中，比如物理块 0 的物理页 0、1、2、3。这时，FTL 的状态如下：



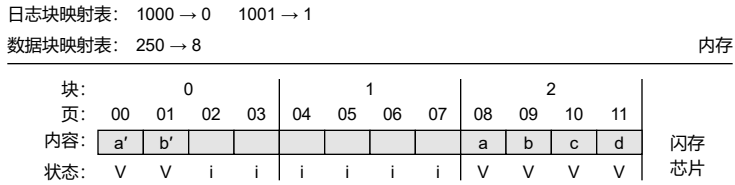
由于这几个块的写入方式和之前完全相同，FTL 可以执行所谓的**交换合并**（switch merge）。在这种情况下，日志块（0）现在成了块 0、1、2、3 的存储位置，由一个块指针

指向；旧块（2）则被擦除，用作日志块（log block）。在这种最好的情况下，原来需要的所有逐页指针都被一个块指针取代了。



交换合并（switch merge）是混合 FTL 的最佳情况。可惜的是，FTL 有时没这么走运。设想这样一种情况：初始条件相同（逻辑块 1000 … 1003 存放在物理块 2 中），但随后客户端覆写的是逻辑块 1000 和 1001。

你觉得这种情况下会发生什么？为什么它更难处理？（先想一想，再看下一页的结果）



为了把这个物理块的其他页重新凑到一起，从而只用一个块指针就能指向它们，FTL 会执行所谓的**部分合并**（partial merge）。在这个操作中，逻辑块 1002 和 1003 从物理块 2 中读出，然后追加到日志（log）中。SSD 最终的状态和上面交换合并（switch merge）的结果相同；但在这种情况下，FTL 必须执行额外的 I/O 才能达到目的，因此增加了写放大（write amplification）。

FTL 遇到的最后一种情况叫作**全合并**（full merge），需要做的工作更多。在这种情况下，FTL 必须从许多其他块中收集页来完成清理。例如，假设逻辑块 0、4、8 和 12 被写入了日志块（log block） A。要把这个日志块（log block）转换成块映射的页，FTL 必须先创建一个包含逻辑块 0、1、2、3 的数据块（data block），因此 FTL 必须从别处读出 1、2、3，再把 0、1、2、3 一起写出。接下来，合并操作还必须对逻辑块 4 做同样的事，找到 5、6、7，把它们合到一个物理块中。逻辑块 8 和 12 也必须这样处理，然后（终于）才能释放日志块（log block） A。不出所料，频繁的全合并（full merge）会严重损害性能，因此应当尽可能避免 [GY+09]。

页映射加缓存

鉴于上面的混合方法很复杂，有人提出了更简单的办法，来降低页映射 FTL（page-mapped FTL）的内存负担。最简单的大概就是只把 FTL 中活跃的部分缓存在内存中，从而减少所需的内存量 [GY+09]。

这种方法的效果可以很好。例如，如果某个工作负载只访问一小部分页，这些页的转换就会存放在内存中的 FTL 里，性能会非常好，内存开销也不高。当然，这种方法也可能表现很差。如果内存放不下所需转换的**工作集**（working set），每次访问至少都需要额外读一次闪存，先把缺失的映射调进来，然后才能访问数据本身。更糟的是，为了给新映射腾出空间，FTL 可能不得不**淘汰**（evict）一个旧映射；如果那个映射是**脏**（dirty）的，也就是还没有持久地写入闪存，还会多出一次写操作。不过，在许多情况下，工作负载会表现出局部性（locality），这种缓存方法既能减少内存开销，又能保持较高的性能。

44.10 磨损均衡

最后，现代 FTL 还必须实现一项相关的后台活动，就是前面介绍过的**磨损均衡**（wear leveling）。基本思想很简单：由于多次擦除/编程循环（erase/program cycle）会让闪存块（flash block）磨损（wear out），FTL 应该尽力把这些工作均匀地分散到设备的所有块上。这样，所有块都会大致在同一时间磨损（wear out），而不是少数几个“热门”块很快就不能用了。

基本的日志结构（log-structured）方法一开始就能很好地分散写负载，垃圾回收也有帮助。不过，有时某个块会被长期存在、不会被覆写的数据填满；在这种情况下，垃圾回收永远不会回收这个块，它也就分不到应有的那一份写负载。

为了解决这个问题，FTL 必须定期把这类块中的所有有效数据读出来，重写到别处，让这个块重新可以写入。磨损均衡（wear leveling）的这个过程增加了 SSD 的写放大（write amplification），从而降低了性能，因为要确保所有块以大致相同的速度磨损（wear out），就需要额外的 I/O。文献中有许多不同的算法 [A+08, M+14]；感兴趣的话可以进一步阅读。

44.11 SSD 的性能和成本

在结束本章之前，我们来考察一下现代 SSD 的性能和成本，以便更好地理解它们在持久存储系统中可能会怎样使用。在这两方面，我们都会和传统的硬盘驱动器比较，并突出两者之间最大的差别。

性能

和硬盘驱动器不同，基于闪存的 SSD 没有机械部件，实际上在许多方面更像 DRAM，因为它们都是“随机访问”设备。和磁盘驱动器相比，性能上最大的差别体现在随机读写上：典型的磁盘驱动器每秒只能执行几百次随机 I/O，而 SSD 能做得好得多。这里我们用一些现代 SSD 的数据，来看看 SSD 的表现到底好多少；我们尤其关心 FTL 把原始芯片的性能问题掩盖得有多好。

图 44.4 给出了三款不同的 SSD 和一款顶级硬盘的一些性能数据；数据取自几个不同的在线来源 [S13, T15]。左边两列是随机 I/O 性能，右边两列是顺序 I/O 性能；前三行是三款不同 SSD（分别来自三星、希捷和英特尔）的数据，最后一行是一块**硬盘驱动器**（hard disk drive，**HDD**）的性能，这里是希捷的一款高端硬盘。

从表中可以了解到几个有趣的事实。第一，也是最惊人的一点，是 SSD 与那块硬盘在随机 I/O 性能上的差距。SSD 的随机 I/O 能达到每秒几十甚至几百 MB，而这块“高性能”硬盘的峰值只有区区几 MB/s（实际上，我们是向上取整才凑到 2 MB/s 的）。第二，可以看到在顺序性能方面，差距要小得多；虽然 SSD 表现更好，但如果你只需要顺序性能，硬盘仍然是不错的选择。第三，可以看到 SSD 的随机读性能不如 SSD 的随机写性能。随机写性能出人意料地好，原因在于许多 SSD 采用日志结构（log-structured）的设计，把随机写转换成了顺序写，从而提高了性能。最后，由于 SSD 的顺序 I/O 和随机 I/O 之间仍有一定的

	随机		顺序	
	读	写	读	写
设备	**(MB/s)**	**(MB/s)**	**(MB/s)**	**(MB/s)**
Samsung 840 Pro SSD	103	287	421	384
Seagate 600 SSD	84	252	424	374
Intel SSD 335 SSD	39	222	344	354
Seagate Savvio 15K.3 HDD	2	2	223	223

图 44.4：**SSD 与硬盘：性能比较**

 © 2008–23, ARPACI-DUSSEAU

性能差异，前面关于如何为硬盘构建文件系统的那些章节中的许多技术，仍然适用于 SSD [AD14b]；虽然顺序和随机 I/O 之间的差距变小了，但仍然大到值得仔细考虑如何设计文件系统、减少随机 I/O。

成本

我们在上面看到，SSD 的性能远远超过现代硬盘，即使是顺序 I/O 也是如此。那么，为什么 SSD 还没有完全取代硬盘，成为首选的存储介质呢？答案很简单：成本，更确切地说，是单位容量的成本。目前 [A15]，一块 250 GB 的 SSD 大约要 150 美元；这样的 SSD 每 GB 要 60 美分。一块典型的 1 TB 硬盘大约 50 美元，也就是每 GB 5 美分。这两种存储介质的成本仍然相差 10 倍以上。

这些性能和成本上的差异，决定了大规模存储系统的构建方式。如果性能是主要考虑因素，SSD 是绝佳的选择，尤其是在随机读性能很重要的时候。反过来，如果你在组建一个大型数据中心，想存储海量信息，巨大的成本差异就会促使你选择硬盘。当然，混合方案也说得通——有些存储系统同时使用 SSD 和硬盘：用数量较少的 SSD 存放更常用的“热”数据，提供高性能，而把其余“较冷”（较少使用）的数据存放在硬盘上以节省成本。只要价格差距还在，硬盘就不会消失。

44.12 小结

基于闪存的 SSD 在笔记本电脑、台式机以及支撑全球经济的数据中心的服务器中越来越常见。所以，你大概应该对它们有所了解，对吧？

坏消息是：本章（和本书的许多章节一样）只是了解当前技术水平的第一步。想进一步了解原始技术，可以看这些资料：关于实际设备性能的研究（例如 Chen 等人 [CK+09] 和 Grupp 等人 [GC+09] 的工作）、FTL 设计中的问题（包括 Agrawal 等人 [A+08]、Gupta 等人 [GY+09]、Huang 等人 [H+14]、Kim 等人 [KK+02]、Lee 等人 [L+07] 和 Zhang 等人 [Z+12] 的工作），甚至由闪存构成的分布式系统（distributed system，包括 Gordon [CG+09] 和 CORFU [B+12]）。另外，恕我们自夸一句，如果想全面了解要从 SSD 榨取高性能都得做哪些事，可以看一篇关于“不成文的契约”的论文 [HK+17]，那里总结得非常好。

不要只读学术论文，也要读读大众媒体上关于最新进展的报道（例如 [V12]）。在那里，你会了解到更实用（但仍然有用）的信息，比如三星在同一块 SSD 中同时使用 TLC 和 SLC 单元，以兼顾性能（SLC 可以快速缓冲写入）和容量（TLC 每个单元能存更多的位）。而这，正如人们常说的，只是冰山一角。自己深入去了解这座研究的“冰山”吧，也许可以从 Ma 等人出色的（而且是最近的）综述 [M+14] 开始。不过要小心：冰山连最强大的船都能击沉 [W15]。

补充：SSD 关键术语

- **闪存芯片**（flash chip）由许多 bank 组成，每个 bank 又组织成**擦除块**（erase block，有时简称**块**，block）。每个块又进一步分成若干**页**（page）。
- 块很大（128 KB–2 MB），包含许多页；页相对较小（1 KB–8 KB）。
- 要从闪存读取，就发出带有地址和长度的读命令；客户端可以借此读取一页或多页。
- 写闪存要复杂一些。首先，客户端必须**擦除**（erase）整个块。然后，客户端可以对每一页**编程**（program），而且只能编程一次，从而完成写入。
- 新的 **trim** 操作可以用来告诉设备某个块不再需要了。
- 闪存的可靠性主要取决于**磨损**（wear out）：如果一个块被擦除和编程得太频繁，它就会无法使用。
- 基于闪存的**固态存储设备**（solid-state storage device，**SSD**）表现得就像一块普通的、基于块读写的磁盘；它借助**闪存转换层**（flash translation layer，**FTL**），把客户端的读写转换成对底层闪存芯片（flash chip）的读取、擦除和编程。
- 大多数 FTL 是**日志结构**（log-structured）的，通过尽量减少擦除/编程循环（erase/program cycle）来降低写入的代价。内存中的转换层记录着逻辑写入在物理介质中的位置。
- 日志结构（log-structured）FTL 的一个关键问题是**垃圾回收**（garbage collection）的代价，它会导致**写放大**（write amplification）。
- 另一个问题是映射表（mapping table）的大小，它可能变得相当大。使用**混合映射**（hybrid mapping），或者只**缓存**（cache） FTL 中的热门部分，都是可能的补救办法。
- 最后一个问题是**磨损均衡**（wear leveling）：FTL 必须时不时把数据从主要被读取的块中迁走，确保这些块也能分担应有的擦除/编程（erase/program）负载。

 © 2008–23, ARPACI-DUSSEAU

参考文献

[A+08] "Design Tradeoffs for SSD Performance" by N. Agrawal, V. Prabhakaran, T. Wobber, J. D. Davis, M. Manasse, R. Panigrahy. USENIX '08, San Diego California, June 2008. *An excellent overview of what goes into SSD design.*

[AD14a] "Operating Systems: Three Easy Pieces" by Chapters: *Crash Consistency: FSCK and Journaling* and *Log-Structured File Systems*. Remzi Arpaci-Dusseau and Andrea Arpaci-Dusseau. *A lot more detail here about how logging can be used in file systems; some of the same ideas can be applied inside devices too as need be.*

[AD14b] "Operating Systems: Three Easy Pieces" by Chapters: *Locality and the Fast File System* and *File System Implementation*. Remzi Arpaci-Dusseau and Andrea Arpaci-Dusseau. *These chapters cover how to build a basic file system for a hard drive. Amazingly, some of these ideas work perfectly well on SSDs! See if you can figure out which design techniques are appropriate, and which are less needed.*

[A15] "Amazon Pricing Study" by Remzi Arpaci-Dusseau. February, 2015. *This is not an actual paper, but rather one of the authors going to Amazon and looking at current prices of hard drives and SSDs. You too can repeat this study, and see what the costs are today. Do it!*

[B+12] "CORFU: A Shared Log Design for Flash Clusters" by M. Balakrishnan, D. Malkhi, V. Prabhakaran, T. Wobber, M. Wei, J. D. Davis. NSDI '12, San Jose, California, April 2012. *A new way to think about designing a high-performance replicated log for clusters using Flash.*

[BD10] "Write Endurance in Flash Drives: Measurements and Analysis" by Simona Boboila, Peter Desnoyers. FAST '10, San Jose, California, February 2010. *A cool paper that reverse engineers flash-device lifetimes. Endurance sometimes far exceeds manufacturer predictions, by up to 100×.*

[B07] "ZFS: The Last Word in File Systems" by Jeff Bonwick and Bill Moore. Available here: http://www.ostep.org/Citations/zfs_last.pdf. *Was this the last word in file systems? No, but maybe it's close.*

[CG+09] "Gordon: Using Flash Memory to Build Fast, Power-efficient Clusters for Data-intensive Applications" by Adrian M. Caulfield, Laura M. Grupp, Steven Swanson. ASPLOS '09, Washington, D.C., March 2009. *Early research on assembling flash into larger-scale clusters; definitely worth a read.*

[CK+09] "Understanding Intrinsic Characteristics and System Implications of Flash Memory based Solid State Drives" by Feng Chen, David A. Koufaty, and Xiaodong Zhang. SIGMETRICS/Performance '09, Seattle, Washington, June 2009. *An excellent overview of SSD performance problems circa 2009 (though now a little dated).*

[G14] "The SSD Endurance Experiment" by Geoff Gasior. The Tech Report, September 19, 2014. Available: http://techreport.com/review/27062. *A nice set of simple experiments measuring performance of SSDs over time. There are many other similar studies; use google to find more.*

[GC+09] "Characterizing Flash Memory: Anomalies, Observations, and Applications" by L. M. Grupp, A. M. Caulfield, J. Coburn, S. Swanson, E. Yaakobi, P. H. Siegel, J. K. Wolf. IEEE MICRO '09, New York, New York, December 2009. *Another excellent characterization of flash performance.*

[GY+09] "DFTL: a Flash Translation Layer Employing Demand-Based Selective Caching of Page-Level Address Mappings" by Aayush Gupta, Youngjae Kim, Bhuvan Urgaonkar. ASPLOS '09, Washington, D.C., March 2009. *This paper gives an excellent overview of different strategies for cleaning within hybrid SSDs as well as a new scheme which saves mapping table space and improves performance under many workloads.*

[HK+17] "The Unwritten Contract of Solid State Drives" by Jun He, Sudarsun Kannan, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. EuroSys '17, Belgrade, Serbia, April 2017. *Our own paper which lays out five rules clients should follow in order to get the best performance out of modern SSDs. The rules are request scale, locality, aligned sequentiality, grouping by death time, and uniform lifetime. Read the paper for details!*

[H+14] "An Aggressive Worn-out Flash Block Management Scheme To Alleviate SSD Performance Degradation" by Ping Huang, Guanying Wu, Xubin He, Weijun Xiao. EuroSys '14, 2014. *Recent work showing how to really get the most out of worn-out flash blocks; neat!*

[J10] "Failure Mechanisms and Models for Semiconductor Devices" by Unknown author. Report JEP122F, November 2010. Available on the internet at this exciting so-called web site: http://www.jedec.org/sites/default/files/docs/JEP122F.pdf. *A highly detailed discussion of what is going on at the device level and how such devices fail. Only for those not faint of heart. Or physicists. Or both.*

[KK+02] "A Space-Efficient Flash Translation Layer For Compact Flash Systems" by Jesung Kim, Jong Min Kim, Sam H. Noh, Sang Lyul Min, Yookun Cho. IEEE Transactions on Consumer Electronics, Volume 48, Number 2, May 2002. *One of the earliest proposals to suggest hybrid mappings.*

[L+07] "A Log Buffer-Based Flash Translation Layer by Using Fully-Associative Sector Translation." Sangwon Lee, Tae-Sun Chung, Dong-Ho Lee, Sangwon Park, Ha-Joo Song. ACM Transactions on Embedded Computing Systems, Volume 6, Number 3, July 2007 *A terrific paper about how to build hybrid log/block mappings.*

[M+14] "A Survey of Address Translation Technologies for Flash Memories" by Dongzhe Ma, Jianhua Feng, Guoliang Li. ACM Computing Surveys, Volume 46, Number 3, January 2014. *Probably the best recent survey of flash and related technologies.*

[S13] "The Seagate 600 and 600 Pro SSD Review" by Anand Lal Shimpi. AnandTech, May 7, 2013. Available: http://www.anandtech.com/show/6935/seagate-600-ssd-review. *One of many SSD performance measurements available on the internet. Haven't heard of the internet? No problem. Just go to your web browser and type "internet" into the search tool. You'll be amazed at what you can learn.*

[T15] "Performance Charts Hard Drives" by Tom's Hardware. January 2015. Available here: http://www.tomshardware.com/charts/enterprise-hdd-charts. *Yet another site with performance data, this time focusing on hard drives.*

[V12] "Understanding TLC Flash" by Kristian Vatto. AnandTech, September, 2012. Available: http://www.anandtech.com/show/5067/understanding-tlc-nand. *A short description about TLC flash and its characteristics.*

[W15] "List of Ships Sunk by Icebergs" by Many authors. Available at this location on the "web": http://en.wikipedia.org/wiki/List_of_ships_sunk_by_icebergs. *Yes, there is a wikipedia page about ships sunk by icebergs. It is a really boring page and basically everyone knows the only ship the iceberg-sinking-mafia cares about is the Titanic.*

[Z+12] "De-indirection for Flash-based SSDs with Nameless Writes" by Yiying Zhang, Leo Prasath Arulraj, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. FAST '13, San Jose, California, February 2013. *Our research on a new idea to reduce mapping table space; the key is to re-use the pointers in the file system above to store locations of blocks, instead of adding another level of indirection.*

作业（模拟）

本节介绍 `ssd.py`，这是一个简单的 SSD 模拟器，可以帮你更好地理解 SSD 是如何工作的。如何运行模拟器，详见 README。这份 README 很长，所以先泡一杯茶（多半得是含咖啡因的），戴好老花镜，让猫蜷在你腿上[1]，然后开始干活。

问题

1. 作业主要关注日志结构（log-structured）的 SSD，用“`-T log`”选项来模拟。其他类型的 SSD 用来作比较。首先，用选项 `-T log -s 1 -n 10 -q` 运行。你能推断出发生了哪些操作吗？用 `-c` 检查你的答案（或者直接用 `-C` 代替 `-q -c`）。用不同的 `-s` 值生成不同的随机工作负载。
2. 现在只显示命令，看看你能否推断出闪存的中间状态。用选项 `-T log -s 2 -n 10 -C` 运行，显示每条命令。然后，确定每两条命令之间闪存的状态；用 `-F` 显示这些状态，看看你对不对。换用不同的随机种子，检验你日渐长进的本事。
3. 我们加上 `-r 20` 选项，让这个问题稍微有意思一点。这会让命令有什么不同？同样用 `-c` 检查你的答案。
4. 性能由擦除、编程和读取的次数决定（这里假设 trim 不花时间）。再次运行上面同样的工作负载，但不显示任何中间状态（例如 `-T log -s 1 -n 10`）。你能估计出这个工作负载需要多长时间才能完成吗？默认的擦除时间为 1000 微秒，编程时间为 40，读取时间为 10。用 `-S` 选项检查你的答案。你还可以用 `-E`、`-W`、`-R` 选项修改擦除、编程和读取的时间。
5. 现在，比较日志结构（log-structured）方法和（非常糟糕的）直接映射（direct-mapped）方法（用 `-T direct` 代替 `-T log`）的性能。先估计一下你认为直接映射（direct-mapped）方法的性能如何，再用 `-S` 选项检查你的答案。一般来说，日志结构（log-structured）方法的性能会比直接映射（direct-mapped）方法好多少？
6. 接下来我们研究垃圾回收器的行为。为此，必须适当设置两条水位线：高水位线（high watermark，`-G`）和低水位线（low watermark，`-g`）。首先，我们观察一下：对日志结构（log-structured）的 SSD 运行一个较大的工作负载，但不做任何垃圾回收，会发生什么。为此，用选项 `-T log -n 1000` 运行；高水位线（high watermark）默认为 10，所以在这种配置下 GC 不会运行。你认为会发生什么？用 `-C`，也许再加上 `-F` 看看。
7. 要打开垃圾回收器，就要使用更小的值。高水位线（high watermark，`-G N`）告诉系统，一旦用掉了 N 个块就开始回收；低水位线（low watermark，`-g M`）告诉系统，一旦只剩 M 个块在使用就停止回收。你认为什么样的水位线值能让系统正常工作？用 `-C` 和 `-F` 显示命令和设备的中间状态，看看结果。

[1]你现在可能会抱怨：“可我是爱狗的人啊！”对此我们只能说：那可真不巧！去弄只猫来，放在腿上，然后做作业！连最基本的指示都不肯照做，你还怎么学东西？

8. 另一个有用的选项是 `-J`，它会显示垃圾回收器运行时在做什么。用选项 `-T log -n 1000 -C -J` 运行，同时查看命令和 GC 的行为。你注意到 GC 有什么特点？当然，GC 最终影响的是性能。用 `-S` 查看最终的统计数据；垃圾回收导致了多少额外的读和写？和理想的 SSD（`-T ideal`）比较一下；由于闪存本身的特性，多出了多少读、写和擦除？再和直接映射（direct-mapped）方法比较一下；日志结构（log-structured）方法在擦除、读取、编程中的哪些方面更胜一筹？

9. 最后一个要研究的方面是**工作负载倾斜**（workload skew）。给工作负载加上倾斜之后，写操作会更多地集中在逻辑块空间中较小的一部分上。例如，用 `-K 80/20` 运行，会让 80% 的写落在 20% 的块上。挑几种不同的倾斜程度，执行大量随机选择的操作（例如 `-n 1000`）：先用 `-T direct` 来理解这种倾斜，再用 `-T log` 看看它对日志结构（log-structured）设备的影响。你预计会发生什么？另一个可以研究的小的倾斜控制选项是 `-k 100`；在有倾斜的工作负载上加上这个选项，前 100 次写入就不会倾斜。这样做的想法是先创建大量数据，然后只更新其中一部分。这可能会对垃圾回收器产生什么影响？